

# Computing Tools Final Exam Study Guide (v2)

Rebuilt to match the professor's final exam announcement exactly. Sections follow his topic list in his order: Git, Docker, AWS, Linux, Terraform, Jenkins, then all 25 practice questions answered. Topics he listed that were never covered in class notes are filled in and marked NEW so you know where the fresh material is.

---

## Read this first

- The announcement is the contract. Every topic below maps to a line in it.
- Several listed topics go BEYOND what class covered: here-documents, CMD vs ENTRYPOINT, Docker networking, S3 versioning, git config scopes, Pipeline post blocks, terraform fmt/output/data blocks/for\_each. Expect questions there. Those sections are flagged NEW.
- His warnings inside the announcement are basically answer keys. Each one is called out below as WATCH OUT.
- Still-standing hints he dropped in class: the AWS search bar, what `lts` means in `jenkins/jenkins:lts`, built-in node executors = 0, "Jenkins organizes when commands run, where they run, and how results are recorded," and the Terraform lifecycle order.

---

## Part 1: Git and GitHub

### Repository states and the staging area

Three places your changes can live:

1. Working directory: where files are created and edited. Git watches it but nothing is saved.
2. Staging area (index): holds the exact changes selected for the next commit.
3. Repository history: completed commits.

File states you must be able to label:

| State     | Meaning                                                                        |
|-----------|--------------------------------------------------------------------------------|
| Untracked | Git has never been told about this file (never added)                          |
| Tracked   | In a previous commit or staged at least once                                   |
| Modified  | Tracked file changed since last commit, not yet staged                         |
| Staged    | Change is in the index, ready for the next commit                              |
| Ignored   | Matches a <code>.gitignore</code> pattern and is untracked, so status hides it |

WATCH OUT (his exact warning): options that automatically include changes to TRACKED files do not include NEW files. `git commit -a -m "msg"` stages and commits modified tracked files only. A brand-new file is untracked, so `-a` skips it entirely. It needs `git add` once first. Same trap: `.gitignore` on a new file works, but a file that's already tracked keeps being tracked.

---

### Examining changes (pick the right command for the right place)

| What you're inspecting                    | Command                                                                                                  |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------|
| General repo status                       | <code>git status</code>                                                                                  |
| Unstaged changes (working dir vs staging) | <code>git diff</code>                                                                                    |
| Staged changes (staging vs last commit)   | <code>git diff --staged</code>                                                                           |
| Ignored files alongside status            | <code>git status --ignored</code>                                                                        |
| Files/changes in a specific commit        | <code>git show &lt;hash&gt;</code> (or <code>git log --stat</code> , <code>git diff hash1 hash2</code> ) |

- The choice depends entirely on WHERE the change lives: working directory, staging area, or history. That's practice question 1.
- `HEAD~1` = one commit before HEAD, so `git diff HEAD~1 HEAD` compares the last two commits.
- Hunk header `@@ -start,count +start,count @@`: first pair is the old file, second the new. First number = starting line, second = span. One number means count is 1.

## Branching and merging

```
git branch          # list branches, * marks current
git branch <name>   # create
git switch <name>   # switch (modern)
git switch -c <name> # create + switch
git checkout -b <name> # older create + switch
git merge <branch>  # merge <branch> INTO the branch you're on
git merge --abort   # cancel an in-progress conflicted merge, back to pre-merge state
```

WATCH OUT (his exact warning): do not assume the branch named in the merge command is the destination. `git merge feature-login` run while on main merges feature-login INTO main. The destination is always the currently checked out branch. The named branch is the SOURCE.

- Current branch: `git branch` (the `*`) or `git status` (first line).
- Fast-forward merge: the current branch has no new commits since divergence, so Git just moves the pointer forward. No merge commit is created.
- Merge commit: both branches advanced, so Git does a three-way merge (common ancestor + both HEADs) and creates a new commit with two parents.
- Conflicts: same lines changed differently on both branches. Git inserts `<<<<<<`, `=====`, `>>>>>>` markers and never guesses. Fix the file manually, `git add` it, `git commit`.
- Aborting: `git merge --abort` returns the branch to its pre-merge state when possible.

## Git configuration scopes (NEW: class only covered --global)

Three scopes, narrowest wins:

| Scope  | Flag                           | File                                 | Applies to                |
|--------|--------------------------------|--------------------------------------|---------------------------|
| System | <code>--system</code>          | <code>/etc/gitconfig</code>          | every user on the machine |
| Global | <code>--global</code>          | <code>~/.gitconfig</code>            | all repos for your user   |
| Local  | <code>--local</code> (default) | <code>.git/config</code> in the repo | that one repository       |

- Local overrides global overrides system. A repo-specific `user.email` (say a work address for one project) overrides your global identity WITHOUT touching any other repo.

- Identity ( `user.name` , `user.email` ) is stamped into every commit you make.
- Which file supplied a setting: `git config --list --show-origin` shows each value with the file it came from. `git config user.email` shows the effective winning value.

## Ignored files

- `.gitignore` patterns hide matching UNTRACKED files from status and keep `git add .` from grabbing them. `*` = any characters, `?` = one character. Rules read top to bottom, last match wins.
- Limitation: ignore rules do NOT remove already-tracked files. Once tracked, always tracked until you untrack it: `git rm --cached <file>` (keeps the local copy), then commit.
- Verify a file matches a rule: `git check-ignore -v <file>` prints the rule and the `.gitignore` line that matched (NEW: not covered in class).
- Show ignored files during review: `git status --ignored`.
- What belongs in `.gitignore`: credentials/secrets, local environment files (`.env`, `venv/`), logs, build artifacts and generated files, `node_modules`. Docker's equivalent is `.dockerignore`.

## Part 2: Docker

### Images, containers, and runtime state

The chain: Dockerfile (build recipe) > image (stacked read-only layers) > container (image + one writable layer on top). Registries like Docker Hub store and distribute images.

- One image can create many independent containers. Each container gets its OWN writable layer.
- WATCH OUT (his framing): changes inside one container live in that container's writable layer only. They do not alter the image and do not appear in any other container started from the same image.
- Overlay view: unchanged files come from read-only layers, edits are copies in the writable layer, deletes are whiteout markers hiding (not destroying) the original.
- Container storage without volumes: `docker stop` keeps the writable layer (you can `start` it again). `docker rm` destroys it permanently.

### Persistent storage: named volumes vs bind mounts

|                            | Named volume                                       | Bind mount                                                                       |
|----------------------------|----------------------------------------------------|----------------------------------------------------------------------------------|
| Who picks the location     | Docker (managed, hidden storage area on host)      | You (an explicit host path)                                                      |
| Portability                | High: no host paths in the command, works anywhere | Low: depends on that exact path existing on that host                            |
| Typical use                | Database data, state that must outlive containers  | Live application source code during development, config files, the Docker socket |
| Survives container removal | Yes                                                | Yes (it's just the host filesystem)                                              |
| Created how                | <code>docker volume create name</code> , or auto   | Nothing to create, path just gets mapped                                         |

```

docker volume create demo-vol
docker run --mount source=demo-vol,destination=/data ubuntu:22.04      # named volume
docker run -v /home/me/app:/app node:20                             # bind mount (host path)
docker run -v /var/run/docker.sock:/var/run/docker.sock jenkins/jenkins:lts # bind of the socket

```

- With `--mount`, a named volume is assumed unless you specify a type. A leading `/` in the `-v` source means bind mount; a bare name means volume.
- Suitability logic for the exam: source code goes in a bind mount (you edit it on the host, container sees changes live). Database data goes in a named volume (Docker manages it, portable, survives redeployment).

## Container networking (NEW: not covered in class)

- Container IPs are assigned dynamically and change when containers are recreated, so hardcoding a container IP into app config is wrong. That's his "why IPs should not be permanent configuration" line.
- User-defined networks: `docker network create mynet`, then `docker run --network mynet --name db ...`. Containers on the same user-defined network can talk to each other; different networks are isolated from each other.
- Docker's internal DNS: on a user-defined network, containers reach each other BY NAME. The app connects to `db:5432`, not an IP. Docker resolves the container name (or Compose service name) to the current IP automatically. This is the fix for the changing-IP problem.
- Internal communication vs publishing: container-to-container traffic on a shared network needs NO published ports. `-p 8080:80` exists only to let the HOST (and the outside world) reach a container. If only other containers need the service, don't publish it.

```

docker network ls
docker network create mynet
docker run -d --network mynet --name db postgres
docker run -d --network mynet --name web -p 8080:5000 myapp # web reaches db as "db"

```

## Dockerfile build behavior and the cache

```

FROM python:alpine
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python", "app.py"]

```

- Each instruction normally creates a layer. On rebuild, Docker reuses cached layers top-down until the FIRST changed instruction (or changed copied file), then re-runs everything after it.
- Build context: the `.` in `docker build -t name .` is the host directory Docker is allowed to see and COPY from. `.dockerignore` excludes files from the context (smaller uploads, and irrelevant files can't invalidate the cache).
- Why dependencies come first: dependency manifests change rarely, source code changes constantly. Copy the manifest, install, THEN copy the code. An `app.py` edit only invalidates the cheap later layers, and the expensive install stays cached. Wrong order = every code edit reinstalls everything. That's practice question 5.
- WATCH OUT: he says study this beyond Python. Same pattern per ecosystem:
- Python: `COPY requirements.txt` then `RUN pip install -r requirements.txt`
- Node: `COPY package.json package-lock.json ./` then `RUN npm install` (or `npm ci`)
- Java/Maven: `COPY pom.xml` then dependency resolution, then the source
- What invalidates a cached layer: editing the instruction line itself, or any change to files referenced by a COPY/ADD (Docker checksums them).

## Container startup: CMD vs ENTRYPOINT (NEW: not covered in class)

- ENTRYPOINT: the fixed executable the container runs. CMD: the default arguments (or, with no entrypoint, the default command).
- When both exist, the effective startup is ENTRYPOINT + CMD appended.
- Arguments after the image name in `docker run image <args>` REPLACE CMD only. The entrypoint still runs, now with your arguments.
- Replacing the entrypoint itself requires the runtime flag: `docker run --entrypoint <executable> image`.

| You want                        | You do                                                  |
|---------------------------------|---------------------------------------------------------|
| Run default app                 | <code>docker run image</code>                           |
| Same entrypoint, different args | <code>docker run image otherargs</code> (overrides CMD) |
| Different executable entirely   | <code>docker run --entrypoint sh image</code>           |

- That distinction is practice question 7: extra args replace the default command (CMD), `--entrypoint` replaces the configured entrypoint.
- Interactive/terminal options: `-i` keeps stdin open, `-t` allocates a tty. Together `-it` gives a usable shell.
- Shell availability: minimal images (alpine, distroless) often lack `/bin/bash`. Try `sh` or `/bin/sh`. Distroless images may have no shell at all.

## Core Docker commands (still fair game)

```
docker version          # client + engine info, proves Docker works
docker run -d --rm -it --name x -p 8080:80 -e VAR=val image:tag
docker pull image:tag    # no tag = :latest (newest, not necessarily stable)
docker images            # local images
docker ps                # running containers
docker ps -a             # all containers, running + stopped
docker stop / start / rm [-f] / rmi
docker logs [--tail N] [-f] <c>
docker inspect <c|image> # inspect the image to find its exposed port
docker exec -it <c> sh    # shell into a RUNNING container
docker build -t name .
```

- `jenkins/jenkins:lts` = Long-Term Support tag, the stable maintained release. He flagged this as final material in class.
- `-f` means follow in `logs -f` but force in `rm -f` / `rmi -f`.

## Part 3: AWS

### The complete request path (his numbered list, memorize the order)

A browser reaching a containerized app on EC2:

- User connects to the instance's PUBLIC address (public IPv4 or public DNS).
- AWS networking routes the request: internet gateway attached to the VPC, route table sending `0.0.0.0/0` to that gateway, instance in a public subnet.
- A security control permits the destination port and protocol: security group inbound rule (protocol, port range, source CIDR).
- The host receives traffic on the published HOST port (and the OS firewall, if any, allows it).

- Docker forwards host port to CONTAINER port per the `-p host:container` mapping.
- The application listens on an address reachable through the container interface: `0.0.0.0`, not `127.0.0.1`. An app bound only to localhost inside the container is unreachable through the mapping.

WATCH OUT (his exact line): a Docker port mapping does not independently authorize AWS network traffic. `-p 8080:80` does nothing if the security group has no inbound rule for 8080. Both layers must allow it. This full chain is practice question 9.

Debugging order when "it works on the instance but not from outside": public IP assigned? > internet gateway + route table? > security group inbound rule (protocol/port/source)? > OS firewall? > correct `-p` mapping? > app listening on 0.0.0.0?

## Remote access to EC2

|              | Linux                                                  | Windows                              |
|--------------|--------------------------------------------------------|--------------------------------------|
| Protocol     | SSH                                                    | RDP                                  |
| Default port | 22                                                     | 3389                                 |
| Credentials  | key pair (.pem), <code>ssh -i key.pem ubuntu@IP</code> | password decrypted with the key pair |
| Client       | terminal ssh / PuTTY (.ppk via PuTTYgen)               | Remote Desktop client                |

- The security group must permit the management port (22 or 3389) from a suitably RESTRICTED source. `0.0.0.0/0` on SSH is the classic bad practice; use your IP `/32`.
- `.pem` is downloadable exactly once. `scp -i key.pem file ubuntu@IP:~` copies files over SSH.

## S3 versioning and deletion (NEW: class covered S3 basics, not versioning mechanics)

- With versioning enabled, every upload of the same key creates a new object VERSION with its own version ID. The newest is the current version; older ones are noncurrent versions.
- Deleting a versioned object does NOT erase data. S3 inserts a DELETE MARKER, which becomes the current version. Ordinary listings hide the object, but every previous version still exists underneath.
- Restore a previous version: delete the delete marker (the old current version resurfaces), or copy a specific noncurrent version back as the new current version.
- Permanently delete: delete a SPECIFIC version by its version ID. That actually removes data.
- Versioning vs backups: versioning protects against overwrites and accidental deletes within the same bucket, but it is not a backup. Deleting the bucket, a misconfigured lifecycle rule, or an attacker with full permissions still loses everything. Backups live somewhere else.

## AWS storage categories (maps to practice question 12)

| Category          | Service | Shape of data                                        | Use it for                                                          |
|-------------------|---------|------------------------------------------------------|---------------------------------------------------------------------|
| Object storage    | S3      | whole objects with metadata, web-addressable by key  | static files, images, backups, anything fetched as a unit           |
| Block storage     | EBS     | raw disk attached to ONE EC2 instance                | the instance's filesystem, databases on that instance, boot volumes |
| Shared filesystem | EFS     | network filesystem mounted by MANY instances at once | shared content several servers read/write together                  |

- Selection logic: independent web-addressable files = S3. A disk for one instance = EBS. A filesystem shared across instances = EFS.

- S3 recap from class: bucket = container, object = file (max 5 TB), bucket names unique in scope, region set at creation. Public access requires BOTH unblocking public-access settings AND a bucket policy (Effect / Principal `*` / Action like `s3:GetObject` / Resource = bucket ARN + `/*` ).

## Broader AWS review (he explicitly says re-study these)

- EC2 fundamentals: instances from AMIs, `t3.micro` = family t, generation 3, size micro. Launch flow ends at the orange button. The AWS console SEARCH BAR is his promised final question for navigating to S3/EC2/IAM/VPC.
- VPC networking: VPC > subnets (public/private) in AZs > route tables decide where traffic goes > internet gateway connects public resources out. Region > AZ > subnet hierarchy; pick regions by customer location; AZs are redundancy.
- Bastion pattern: SSH to the public instance, hop to the private instance by private IP. Keeps databases off the internet.
- Security groups: per-instance virtual firewall, inbound + outbound, protocol + port + source CIDR. `0.0.0.0/0` anywhere, `/32` one address.
- IAM: users (people or applications), groups (permissions at scale), roles (temporary permissions, often for services), policies (JSON allow/deny documents). Root user secured with MFA, daily work through limited users. Least privilege: grant only what's needed, request more later.
- Persistent EC2 storage = EBS volumes (block storage above).

## Part 4: Linux

### Environment variables and command lookup

- Environment variables are named values a process holds; child processes INHERIT the parent shell's exported variables.
- Display one: `echo $VAR` ( `$` dereferences). List all: `env` .
- Temporary set (current shell only): `VAR=value` . Export to children: `export VAR=value` . Either lasts only for the session.
- Permanent: put the export in shell startup config ( `~/.bashrc` , `~/.profile` ), which runs on every new shell.
- Command lookup: type a bare command name and the shell searches the directories in `$PATH` , colon-separated, IN ORDER, left to right, running the FIRST match found.
- Risk of unsafe PATH entries (NEW angle): putting `.` or a world-writable directory early in PATH means a malicious file named `ls` in the current directory runs instead of the real `ls` . Order matters, and attackers exploit it.

### Standard streams and redirection

| Stream | FD |
|--------|----|
| STDIN  | 0  |
| STDOUT | 1  |
| STDERR | 2  |

```
command > out.txt           # redirect stdout (overwrite), shorthand for 1>
command >> out.txt           # append
command 2> err.txt          # redirect stderr
command > all.txt 2>&1       # both streams to one file
command &> all.txt           # modern shortcut for the same
command > out.txt 2> err.txt # keep streams in SEPARATE files
command 2>/dev/null         # throw errors away
cmd1 | cmd2                 # pipe stdout into next command
cmd1 |& cmd2                 # pipe both streams
```

WATCH OUT: redirection order matters, and it's practice question 13. Redirections apply LEFT TO RIGHT:

- `command > output.log 2>&1` : first stdout goes to the file, THEN stderr is pointed at wherever stdout currently points (the file). Both end up in output.log. Correct.
- `command 2>&1 > output.log` : first stderr is pointed at wherever stdout currently points (the terminal), THEN stdout is moved to the file. Result: stdout in the file, stderr still on the terminal.

`2>&1` means "duplicate stderr to stdout's CURRENT target," not "tie them together forever."

## Here-documents (NEW: not covered in class)

A here-document feeds a multiline block of input to a command without an editor:

```
cat << EOF > config.txt
server=localhost
port=8080
user=$USER

---

# Part 3: AWS

## The complete request path (his numbered list, memorize the order)

A browser reaching a containerized app on EC2:

1. User connects to the instance's PUBLIC address (public IPv4 or public DNS).
2. AWS networking routes the request: internet gateway attached to the VPC, route table sending `0.0.0.0/0` to that gateway, instance
3. A security control permits the destination port and protocol: security group inbound rule (protocol, port range, source CIDR).
4. The host receives traffic on the published HOST port (and the OS firewall, if any, allows it).
5. Docker forwards host port to CONTAINER port per the `-p host:container` mapping.
6. The application listens on an address reachable through the container interface: `0.0.0.0`, not `127.0.0.1`. An app bound only to `

WATCH OUT (his exact line): a Docker port mapping does not independently authorize AWS network traffic. `-p 8080:80` does nothing if `

Debugging order when "it works on the instance but not from outside": public IP assigned? > internet gateway + route table? > security

## Remote access to EC2

| | Linux | Windows |
|---|---|---|
| Protocol | SSH | RDP |
| Default port | 22 | 3389 |
| Credentials | key pair (.pem), `ssh -i key.pem ubuntu@IP` | password decrypted with the key pair |
| Client | terminal ssh / PuTTY (.ppk via PuTTYgen) | Remote Desktop client |

- The security group must permit the management port (22 or 3389) from a suitably RESTRICTED source. `0.0.0.0/0` on SSH is the classic
- `.pem` is downloadable exactly once. `scp -i key.pem file ubuntu@IP:~` copies files over SSH.

## S3 versioning and deletion (NEW: class covered S3 basics, not versioning mechanics)

- With versioning enabled, every upload of the same key creates a new object VERSION with its own version ID. The newest is the current
- Deleting a versioned object does NOT erase data. S3 inserts a DELETE MARKER, which becomes the current version. Ordinary listings h
- Restore a previous version: delete the delete marker (the old current version resurfaces), or copy a specific noncurrent version ba
- Permanently delete: delete a SPECIFIC version by its version ID. That actually removes data.
- Versioning vs backups: versioning protects against overwrites and accidental deletes within the same bucket, but it is not a backup

## AWS storage categories (maps to practice question 12)

| Category | Service | Shape of data | Use it for |
```



```

|---|---|---|---|
| Object storage | S3 | whole objects with metadata, web-addressable by key | static files, images, backups, anything fetched as a un
| Block storage | EBS | raw disk attached to ONE EC2 instance | the instance's filesystem, databases on that instance, boot volumes |
| Shared filesystem | EFS | network filesystem mounted by MANY instances at once | shared content several servers read/write together

- Selection logic: independent web-addressable files = S3. A disk for one instance = EBS. A filesystem shared across instances = EFS.
- S3 recap from class: bucket = container, object = file (max 5 TB), bucket names unique in scope, region set at creation. Public acco

## Broader AWS review (he explicitly says re-study these)

- EC2 fundamentals: instances from AMIs, `t3.micro` = family t, generation 3, size micro. Launch flow ends at the orange button. The r
- VPC networking: VPC > subnets (public/private) in AZs > route tables decide where traffic goes > internet gateway connects public r
- Bastion pattern: SSH to the public instance, hop to the private instance by private IP. Keeps databases off the internet.
- Security groups: per-instance virtual firewall, inbound + outbound, protocol + port + source CIDR. `0.0.0.0/0` anywhere, `/32` one s
- IAM: users (people or applications), groups (permissions at scale), roles (temporary permissions, often for services), policies (JSO
- Persistent EC2 storage = EBS volumes (block storage above).

---

# Part 4: Linux

## Environment variables and command lookup

- Environment variables are named values a process holds; child processes INHERIT the parent shell's exported variables.
- Display one: `echo $VAR` (`$` dereferences). List all: `env`.
- Temporary set (current shell only): `VAR=value`. Export to children: `export VAR=value`. Either lasts only for the session.
- Permanent: put the export in shell startup config (`~/.bashrc`, `~/.profile`), which runs on every new shell.
- Command lookup: type a bare command name and the shell searches the directories in `$PATH`, colon-separated, IN ORDER, left to right
- Risk of unsafe PATH entries (NEW angle): putting `.` or a world-writable directory early in PATH means a malicious file named `ls` :

## Standard streams and redirection

| Stream | FD |
|---|---|
| STDIN | 0 |
| STDOUT | 1 |
| STDERR | 2 |

```bash
command > out.txt          # redirect stdout (overwrite), shorthand for 1>
command >> out.txt          # append
command 2> err.txt         # redirect stderr
command > all.txt 2>&1      # both streams to one file
command && all.txt          # modern shortcut for the same
command > out.txt 2> err.txt # keep streams in SEPARATE files
command 2>/dev/null        # throw errors away
cmd1 | cmd2                # pipe stdout into next command
cmd1 |& cmd2                # pipe both streams

```

WATCH OUT: redirection order matters, and it's practice question 13. Redirections apply LEFT TO RIGHT:

- `command > output.log 2>&1` : first stdout goes to the file, THEN stderr is pointed at wherever stdout currently points (the file). Both end up in output.log. Correct.
- `command 2>&1 > output.log` : first stderr is pointed at wherever stdout currently points (the terminal), THEN stdout is moved to the file. Result: stdout in the file, stderr still on the terminal.

`2>&1` means "duplicate stderr to stdout's CURRENT target," not "tie them together forever."

## Here-documents (NEW: not covered in class)

A here-document feeds a multiline block of input to a command without an editor:

```
cat << MYEOF > config.txt
server=localhost
port=8080
user=$USER
MYEOF
```

- `<< DELIM` opens it; the block runs until a line containing ONLY the delimiter, at the start of the line, spelled exactly. Misplaced or misspelled closing delimiter = the shell keeps reading forever.
- Add `> file` to write the block into a file (the exam skill: build a short config file with no interactive editor).
- Unquoted delimiter ( `<< MYEOF` ): variables EXPAND, so `$USER` becomes your username.
- Quoted delimiter ( `<< 'MYEOF'` ): everything stays literal, `$USER` remains the text `$USER`.
- You've seen this pattern in AWS user-data scripts and setup snippets.

## Symbolic links and path resolution

- `ln -s <target> <link_name>` : first argument is what you point at, second is the link you create. `l` as the first character of `ls -l` output.
- Logical path: the path you NAVIGATED, preserving the symlink. `pwd -L` (the shell's default).
- Physical path: the real filesystem destination with symlinks resolved. `pwd -P`.
- Two displayed paths can refer to the same physical directory: enter through `current` (a symlink to `releases/v2.3`) and `pwd -L` says `.../current` while `pwd -P` says `.../releases/v2.3`. Both are correct; they answer different questions. That's practice question 14.
- Hard links share an inode (same file, another name). No hard links to directories except the automatic `.` and `..`. Empty directory link count = 2; each subdirectory adds 1 to the parent via its `..`.

## Directory navigation

```
pwd          # current directory
pwd -L / -P  # logical vs physical
cd ~         # home (also plain cd)
cd ..        # parent
cd /etc/nginx # absolute path (starts with /)
cd docs/notes # relative path (from where you are)
cd -         # PREVIOUS directory (the last one you were in, not the parent)
```

- `cd -` is practice question 15: bounce between two distant directories with no retyping. It also prints where it took you.
- `.` = current directory, `..` = parent, both auto-created hard links.

# Part 5: Terraform

## Core workflow (expanded: he added fmt and output)

Write config, then:

| Stage      | Command                         | What it does                                                                                                                                                                                                    |
|------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Initialize | <code>terraform init</code>     | downloads provider plugins per required_providers, creates the <code>.terraform</code> directory and the dependency lock file <code>.terraform.lock.hcl</code> . Re-run after adding a provider. Safe to repeat |
| Format     | <code>terraform fmt</code>      | rewrites .tf files into canonical style (indentation, alignment). Changes formatting only, never behavior                                                                                                       |
| Validate   | <code>terraform validate</code> | checks syntax and internal consistency of the config. No cloud calls, no changes                                                                                                                                |
| Preview    | <code>terraform plan</code>     | computes proposed changes against state. Changes nothing                                                                                                                                                        |
| Apply      | <code>terraform apply</code>    | executes the changes, prompts for <code>yes</code> (skip with <code>-auto-approve</code> )                                                                                                                      |
| Inspect    | <code>terraform output</code>   | prints the configuration's output values after apply                                                                                                                                                            |
| Destroy    | <code>terraform destroy</code>  | tears down everything Terraform manages, prompts for <code>yes</code>                                                                                                                                           |

- Exam framing: "which command at this point in the workflow, and what does it NOT change." `fmt` and `validate` never touch infrastructure. `plan` never touches infrastructure. `init` touches only the local working directory.
- `.terraform` directory: local working data, mainly installed provider plugins. `.terraform.lock.hcl`: pins exact provider versions for reproducibility.
- State: `terraform.tfstate` records what exists (that's how `plan` computes changes), `terraform.tfstate.backup` is one iteration behind. Never hand-edit state. `terraform state list`, `terraform state show <address>`, `terraform show`.

## Block types (know the role of each)

---

```

terraform {                                # settings + required_providers
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"                # >= 3.0, < 4.0
    }
  }
}

provider "docker" {}                      # configures the provider (auth, region, endpoints)

resource "docker_image" "nginx" {         # resource: something Terraform CREATES and manages
  name = "nginx:latest"
}

data "aws_ami" "ubuntu" {                # data block: READS existing infrastructure, creates nothing
  most_recent = true
}

variable "app_name" {                    # input variable
  type     = string
  default = "flask-app"
}

locals {                                  # local values, referenced as local.name
  common_tags = { project = "demo" }
}

output "url" {                           # output value, printed after apply / terraform output
  value = "http://localhost:${var.external_port}"
}

```

- Provider vs resource vs data source (he lists this distinction explicitly): a PROVIDER is the plugin that talks to a platform's API. A RESOURCE is an object Terraform creates and manages through that provider. A DATA SOURCE reads something that already exists so you can reference it, without managing it.
- Variable-definition files: `terraform.tfvars` holds `name = value` assignments mapped onto declared variables. No value anywhere = default is used.
- References: `var.app_name`, `local.common_tags.project`, `docker_image.nginx.image_id`, `data.aws_ami.ubuntu.id`. Interpolation in strings: `"${var.app_name}-dev"`.
- File names are convention only; Terraform merges every `.tf` file in the directory.

## Reading a plan (he added replacement)

| Symbol           | Meaning                                                                                   |
|------------------|-------------------------------------------------------------------------------------------|
| <code>+</code>   | resource will be created                                                                  |
| <code>~</code>   | updated IN PLACE, resource survives                                                       |
| <code>-</code>   | destroyed                                                                                 |
| <code>-/+</code> | REPLACED: destroyed then recreated because a changed attribute can't be modified in place |

- Replacement vs in-place is practice question 16: in-place keeps the object (and its data/identity), replacement destroys and recreates it, so anything stored only in it is lost and IDs/IPs can change. The plan marks the attribute forcing replacement.
- "known after apply": values that can't exist until creation (IDs, IPs).

- WATCH OUT (his line): don't rely only on the final change summary ("3 to add, 1 to change"). Read WHICH attributes caused each proposed action.

## Resource addresses (expanded: modules and indexes)

- Base form: `type.local_name`, e.g. `docker_container.nginx`. The type comes from the provider. The local name is just YOUR label. It does not have to match any external name (the container's real name comes from the `name` argument).
- Attributes hang off the address: `docker_container.nginx.id`.
- Module paths prefix the address: `module.network.aws_subnet.public`.
- Indexed resources (count): `aws_instance.web[0]`, `aws_instance.web[1]`.
- for\_each resources use the KEY as the index: `aws_instance.web["prod"]`.
- WATCH OUT (his line): never assume the local Terraform name matches the infrastructure object's external name. The quiz already tested exactly this with `docker_container.nginx`.

## Dependencies

- Implicit: created by an attribute REFERENCE. `image = docker_image.nginx.image_id` makes the container depend on the image; Terraform builds the graph and orders creation automatically.
- Explicit: `depends_on = [docker_container.mongodb]` for orderings Terraform can't infer (class example: MongoDB before Mongo Express).
- When explicit is justified: only when a real dependency exists that no attribute reference expresses.
- WATCH OUT (his line): unnecessary `depends_on` reduces PARALLELISM. Independent resources are created concurrently; fake dependencies serialize them and slow applies.
- Textual order inside a .tf file controls NOTHING. Only the graph decides provisioning order.

## Saved plans (maps to practice question 19)

```
terraform plan -out=planfile
terraform apply planfile          # applies EXACTLY what was reviewed
```

- Applying a saved plan executes precisely the reviewed changes: nothing recomputed, no surprises. That's why it's safer than a fresh unreviewed plan in approval/automation workflows. He called it a contract.
- Why a saved plan can become unusable: it was computed against a snapshot of state and config. If infrastructure drifted or configuration changed since Monday, the plan is stale and Terraform refuses or the plan no longer matches reality. Old plans expire in usefulness.
- `terraform apply` WITHOUT a file generates a brand-new plan and asks for approval right then. Know the difference.

## Providers

- Provider plugins are downloaded during `terraform init` based on the `required_providers` block (source like `kreuzwerker/docker` or `hashicorp/aws`, plus a version constraint).
- Version constraints: `~> 3.0` allows 3.x but not 4.0. Pinning prevents new provider releases from breaking config. The lock file records the exact resolved version.
- The `provider` block carries configuration: authentication, region, endpoints (AWS needs region + credentials; Docker often needs nothing locally).
- What a provider actually does: translates your HCL into the platform's API calls. Terraform core plans; the provider executes against Docker/AWS/etc.
- Registry docs (registry.terraform.io) define each resource's required and optional arguments and exported attributes.

## Part 6: Jenkins

### Controller and agent architecture (be able to assign every duty)

| Responsibility                      | Component                                           |
|-------------------------------------|-----------------------------------------------------|
| Stores job and Pipeline definitions | Controller                                          |
| Provides the web interface          | Controller                                          |
| Schedules work / assigns builds     | Controller                                          |
| Manages users, credentials, plugins | Controller                                          |
| Executes build commands             | Agent                                               |
| Maintains the build workspace       | Agent (the one running the build)                   |
| Returns logs and results            | Agent reports back, controller records and displays |

- Why production separates them (his line + practice question 22): a controller doing everything (UI, scheduling, user management, AND compiling apps, AND building images) starves itself. Builds are heavy and untrusted; running them on the controller is a stability and SECURITY risk (build code executing on the box that holds all credentials). Agents isolate the heavy, risky work; the controller stays a lightweight brain.
- Class-flagged best practice: set the built-in node's executors to 0 so nothing ever builds on the controller. Only after agents exist, or every build hangs with nowhere to run.
- Executors = concurrent builds a node can run. Labels = how jobs select agents. Permanent agents stay up; cloud agents (Docker) spin up per build and vanish. The Docker socket mount is what lets a containerized Jenkins create agent containers.
- Core vocabulary: job (saved automation definition), build (one execution), workspace (job's working directory), plugin (extension), node (any machine that can execute builds; the built-in node IS the controller).
- The flagged definition: Jenkins organizes WHEN commands run, WHERE they run, and HOW the result is recorded.

### Build triggers (he added upstream jobs)

| Trigger            | Behavior                                                            | Use when                                                         |
|--------------------|---------------------------------------------------------------------|------------------------------------------------------------------|
| Manual             | human clicks Build Now                                              | testing, one-offs                                                |
| Webhook            | repo host pushes an event to Jenkins, build starts immediately      | preferred; Jenkins is reachable from the repo host               |
| Poll SCM           | Jenkins checks the repo on a schedule, builds ONLY if changes exist | inbound webhooks can't reach Jenkins (private IP, NAT, firewall) |
| Build periodically | runs on a cron schedule UNCONDITIONALLY, code changes irrelevant    | maintenance windows, nightly tasks                               |
| Upstream job       | build starts when another job finishes                              | chained jobs: deploy after build succeeds                        |

- WATCH OUT (his exact distinction): Poll SCM checks a repository on a schedule and may do nothing. Build periodically executes on the schedule no matter what. That plus webhooks is practice question 23.
- Why polling exists: a machine behind NAT/private IP can't receive inbound webhook traffic, so it reaches OUT on an interval instead.

## Freestyle job configuration sections (classify settings into sections)

| Section                | What belongs there                                           |
|------------------------|--------------------------------------------------------------|
| Source Code Management | repository URL, branch, credentials for private repos        |
| Build Triggers         | webhook, poll SCM, periodic schedule, upstream job           |
| Build Environment      | workspace prep, e.g. delete workspace before build starts    |
| Build Steps            | the actual commands: Execute shell, etc.                     |
| Post-build Actions     | notifications, publishing artifacts, cleanup after the build |

- Expect a matching question: "where do repository credentials go" (SCM), "where does the shell command go" (Build Steps), "where do email notifications go" (Post-build Actions).
- Freestyle = point-and-click jobs, good for simple scripts and debugging. Pipelines are what production uses.

## Declarative Pipeline organization (NEW: class named pipelines, never the syntax)

```
pipeline {
  agent { label 'docker-agent' } // where the whole Pipeline (or a stage) runs
  stages {
    stage('Checkout') {
      steps { git 'https://github.com/user/repo.git' }
    }
    stage('Test') {
      steps { sh 'pytest' }
    }
    stage('Package') {
      steps { sh 'docker build -t app .' }
    }
  }
  post {
    always { sh 'docker rm -f test-db || true' } // cleanup no matter what
    success { echo 'good' }
    failure { echo 'broken' }
  }
}
```

Hierarchy, outermost in: `pipeline` (the whole definition) > `agent` (where it runs) > `stages` (the ordered container) > `stage` (one named phase) > `steps` (the executable block) > individual steps (`sh`, `git`, `echo`). `post` sits alongside stages.

- A stage organizes a meaningful PHASE; executable operations belong inside `steps`. His line verbatim.
- Why stages beat one giant block (practice question 24): each stage reports its own pass/fail status and timing in the UI, so you see WHERE it broke (Checkout vs Test vs Package) without reading the whole log. Stages can also target different agents and gate later phases.

## Post-run behavior (NEW)

| Condition             | Runs                                                                 |
|-----------------------|----------------------------------------------------------------------|
| <code>always</code>   | after every result, unconditional                                    |
| <code>success</code>  | only if the build succeeded                                          |
| <code>failure</code>  | only if it failed                                                    |
| <code>changed</code>  | only when the result differs from the previous build                 |
| <code>unstable</code> | when the build is unstable (e.g. test failures without a hard error) |
| <code>cleanup</code>  | last, after every other post condition, for final teardown           |

- `post` blocks can sit at the Pipeline level or inside an individual stage.
- Why unconditional cleanup matters (practice question 25): a "Cleanup" STAGE at the end never runs if an earlier stage fails, because a failed stage stops the stage sequence. Temporary containers then leak. `post { always { ... } }` (or `cleanup`) runs regardless of failure, so teardown is guaranteed.

## Jenkins setup recap (still fair game from class)

```

sudo apt update && sudo apt upgrade -y
sudo apt install docker.io -y
sudo systemctl enable --now docker      # enable = start on boot, --now = start immediately
sudo usermod -aG docker $USER           # -a append, -G group
newgrp docker                           # apply group without logout

sudo docker volume create jenkins_home
sudo docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts

docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

```

- 8080 = web UI (security group needs the inbound rule), 50000 = agent communication, named volume = survives container removal, socket mount = Jenkins can drive the host's Docker, `lts` = Long-Term Support. Initial password is shown once and deleted after setup.

## Part 7: All 25 practice questions, answered

**1. Unstaged vs staged changes: what distinction matters?** Where the changes live. Changes not yet staged sit in the working directory, inspected with `git diff`. Changes selected for the next commit sit in the staging area, inspected with `git diff --staged`. Same command family, different comparison target: working-vs-staging versus staging-vs-last-commit.

**2. Command showing the current state of the working directory and staging area?** `git status`.

**3. Command staging all current changes in the repository directory?** `git add .` from the repo root (or `git add -A` for the whole repo regardless of location).

**4. Purpose of a Git commit?** To permanently record the staged changes as a snapshot in the repository history, with an author, timestamp, message, and a pointer to its parent commit.



**5. Rebuild reinstalls dependencies on every source change. Fix via Dockerfile structure?** Order the Dockerfile so the dependency manifest is copied and installed BEFORE the application source: `COPY requirements.txt .` then `RUN pip install -r requirements.txt` then `COPY . .`. Docker caches layers top-down and invalidates only from the first change onward. Source edits then invalidate only the cheap final COPY, and the expensive install layer stays cached. It reruns only when the manifest itself changes.

**6. Command displaying locally stored images?** `docker images`.

**7. Replace the default command vs replace the entrypoint?** Arguments after the image name in `docker run image <args>` replace only CMD, the image's default arguments/command; the configured ENTRYPOINT still runs and receives them. To run a different executable entirely, override the entrypoint at runtime: `docker run --entrypoint <diagnostic> image`. CMD = default arguments, ENTRYPOINT = the fixed executable.

**8. What does VPC stand for, and its purpose?** Virtual Private Cloud: your isolated private network inside AWS, the container that EC2 instances and other resources live in, subdivided into subnets with route tables controlling traffic.

**9. Works on the instance, unreachable from outside. Layers to examine, AWS down to the process?** (1) Instance has a public IPv4/DNS. (2) VPC routing: internet gateway attached, route table sends 0.0.0.0/0 to it, instance in a public subnet. (3) Security group inbound rule allows the protocol and port from the client's source. (4) OS firewall on the host allows the port. (5) Docker port mapping publishes host port to container port ( `-p 8080:80` ). (6) The application listens on 0.0.0.0 inside the container, not 127.0.0.1.

**10. Linux vs Windows EC2 remote management?** Linux: SSH, port 22, key-pair authentication with the .pem file, terminal client ( `ssh -i key.pem ubuntu@IP` ). Windows: RDP, port 3389, Administrator password decrypted using the key pair, Remote Desktop client with a GUI. Each needs its management port opened in the security group from a restricted source.

**11. What does IAM stand for?** Identity and Access Management: who can access AWS resources and what actions they may perform, via users, groups, roles, and JSON policies.

**12. Storage category for each requirement?** Web-addressable independent objects: object storage, S3. Disk attached to one EC2 instance: block storage, EBS. Shared filesystem mounted by several instances: EFS.

**13. `command > output.log 2>&1` vs `command 2>&1 > output.log` ?** Redirections apply left to right. First form: stdout is pointed at the file, then stderr is duplicated to stdout's current target, the file. Both streams land in output.log. Second form: stderr is duplicated to stdout's current target, still the terminal, and only then is stdout moved to the file. Result: stdout in the file, stderr on the terminal. `2>&1` copies stderr to wherever stdout points AT THAT MOMENT.

**14. Symlink `current` vs the reported release path?** Both are correct because they describe the same physical directory two ways. The logical path ( `pwd -L` , what the shell preserves) shows the route taken, through the `current` symlink. The physical path ( `pwd -P` , what the diagnostic resolved) shows the real destination, the versioned release directory. The symlink is a pointer, so two names refer to one location.

**15. Return to the last directory without retyping the path?** `cd -` switches to the previously visited directory (tracked in \$OLDPWD) and prints it. Alternating `cd -` bounces between the two locations.

**16. Update in place vs replacement?** In place ( `~` ): the existing resource is modified and survives, keeping its identity and data. Replacement ( `-/+` ): the resource is destroyed and recreated because a changed attribute cannot be altered on a live object. Anything stored only inside it is lost, and identifiers like IPs can change, so replacement carries downtime and data risk.

**17. Terraform config file extension?** `.tf` (with `.tfvars` for variable-definition files).

**18. Command removing infrastructure managed by the current configuration?** `terraform destroy`.

**19. Why apply Monday's saved plan on Friday, and why might it fail?** Safer: the saved plan is the exact reviewed and approved set of changes. Applying the file executes precisely that, while a fresh Friday plan could include unreviewed differences. Might no longer be usable: the plan was computed against Monday's state and config. If infrastructure drifted or configuration changed during the week, the plan is stale and no longer matches reality, so Terraform rejects it.

**20. What is a Jenkins controller?** The central Jenkins server: stores job and Pipeline definitions, serves the web UI, schedules and assigns builds, and manages users, credentials, and plugins. It coordinates; it should not execute builds.

**21. Purpose of a Jenkins agent?** To execute the actual build work: run the commands, maintain the build workspace, and report logs and results back to the controller.

**22. Overloaded central server: how does controller-agent help?** Move build execution off the controller onto agents. The controller keeps the lightweight coordination duties (UI, users, scheduling, credentials) while agents do the heavy

compiling and image building, possibly in parallel across machines or on-demand cloud/Docker agents. This restores responsiveness, scales horizontally, and removes the security risk of untrusted build code running on the machine holding all credentials. Best practice: built-in node executors set to 0 once agents exist.

**23. Trigger strategy per situation?** Build immediately after a GitHub push: webhook, GitHub notifies Jenkins the moment the push happens, no delay, no wasted checks. Jenkins can't receive inbound requests: Poll SCM, Jenkins reaches out on a schedule and builds only when it finds changes, which works behind NAT/private IPs. Every Sunday regardless of changes: Build periodically with a cron schedule, since the run is unconditional and unrelated to commits.

**24. Why stages beat one big block?** Separate stages report individual status, timing, and logs in the UI, so a failure shows exactly which phase broke (Checkout vs Test vs Package) without digging through one giant log. Stages also gate progression (no packaging if tests fail), can run on different agents, and make the workflow readable and maintainable.

**25. Cleanup of temporary containers that must run even on failure?** Put the removal commands in a `post` block with the `always` (or `cleanup`) condition, at the Pipeline level or in the relevant stage. A normal final "Cleanup" stage is unreliable because any earlier stage failure stops the stage sequence and the cleanup stage never runs, leaking containers. `post { always { ... } }` executes regardless of the result.

---

## The night-before sheet

- Merge destination = the branch you're ON. `commit -a` skips untracked files. Local config beats global beats system. `git check-ignore -v` proves a rule matches. `git rm --cached` untracks without deleting.
- Volumes: Docker-managed, portable, databases. Bind mounts: your host path, source code. Containers on the same user-defined network talk BY NAME, no published ports needed. `-p` is for the host/world only.
- Extra `docker run` args replace CMD. `--entrypoint` replaces ENTRYPOINT. Dependencies before source in the Dockerfile, in every ecosystem.
- Request path: public IP > IGW/route table > security group > OS firewall > `-p` mapping > app on 0.0.0.0. Docker mapping never authorizes AWS traffic.
- S3 delete on a versioned bucket = delete marker, data intact. S3 objects, EBS one-instance disk, EFS shared filesystem.
- `> f 2>&1` both to file. `2>&1 > f` stderr stays on terminal. Heredoc: quoted delimiter = no expansion. `pwd -L` route, `pwd -P` reality. `cd -` bounces.
- init, fmt, validate, plan, apply, output, destroy. `-/+` = replace. Local name ≠ real name. References make implicit dependencies; needless `depends_on` kills parallelism. Saved plan = reviewed contract, but it goes stale.
- Controller thinks, agents build, executors 0 on the built-in node. Poll SCM checks and maybe builds; periodic always builds. Cleanup lives in `post { always }`, never in a final stage.
- Free points he promised: AWS search bar, `lts` = Long-Term Support, "Jenkins organizes when, where, and how."